Wiederholungsklausur Informatik I
Sommersemester 2007
Prof. Dr. Carsten Damm
19. April 2007, 12:00 - 13:30 Uhr
Bearbeitungszeit: 90 Minuten

Vorname:

Nachname:

Matrikelnummer:

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

	Punkte	Schätzung für
Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung)	20	10
Aufgabe 2 (Aufwand und Komplexität + 3 Bonuspunkte)	15	10
Aufgabe 3 (Objektorientierung & Programmierung)	10	5
Aufgabe 4 (Java)	20	10
Aufgabe 5 (Sortierverfahren)	10	5
Aufgabe 6 (Algorithmen und Datenstrukturen)	15	8
Summe	90	48

Note:

Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung [20 Punkte])

a) Geben Sie einen regulären Ausdruck über dem Alphabet $\{a, f, g, s, v, z\}$ an, der das übliche Verhalten von Passagieren in einem Flughafen beschreibt.

Die Aktionen sind: Gepäckwagen nehmen: g, Gepäckwagen zurückgeben: z, Flughafen verlassen: v, Gepäckwagen schieben: s, Gepäckstück am Schalter abgeben: a, mit Flugzeug abreisen: f.

Beispiel $gsssaaaszf$: Passagier schiebt Gepäckwagen mehrfach, gibt drei Gepäckstücke am Schalter ab, gibt den Gepäckwagen ab und fliegt ab.

Vorgaben: Eingang, Ausgang und Schalter sind voneinander entfernt. Wenn Gepäckstücke am Schalter nicht abgegeben werden (Reisepaß abgelaufen), wird der Gepäckwagen zurückgegeben und der Flughafen verlassen. (6 P)

Lösung

$gss^*aa^*ss^*zf \mid gss^*zv \;=\; gss^*(aa^*ss^*zf \mid zv)$

b) Entwerfen Sie einen endlichen Automaten der die folgende Sprache L über dem Alphabet $\{1, 2, ., E\}$ akzeptiert (Zustandsdiagramm genügt).

$$L = (\{1,2\}\{1,2\}^*.\{1,2\}^*)(E\{1,2\}\{1,2\}^* \cup \{\varepsilon\})$$

Erläuterung: L besteht aus den Gleitkommaliteralen, die mit einer Ziffer beginnen, einen Dezimalpunkt enthalten und außer diesem und einem eventuellen Exponentenzeichen nur die Ziffern 1 und 2 (z.B. 12.12 oder 12.E2, aber nicht 123.123 oder .11E12 oder 12.E-1): (6 P)

Lösung

Siehe Automat auf Beiblatt 1

c) Gegeben sind die Zahlen 128 und 35. Ihre Summe ist 163.

1. Stellen Sie beide als 8-Bit Binärzahlen dar. (3 P)

   Lösung

   128: 10000000 (1.5 P)
   35: 00100011 (1.5 P)

2. Addieren Sie beide Zahlen im Binärformat und verifizieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (2 P)

   Lösung

	128:	10000000	
	35:	00100011	
Add:	163:	10100011	= 128 + 32 + 2 + 1

3. Stellen Sie -35 als Zweierkomplement dar und berechnen Sie $128 - 35$. (3 P)

Lösung

Zweierkomplement von 35: 35 ist		00100011
Einerkomplement:		11011100
(Zweierkomplement) +1:		11011101
Addieren:	10000000	
+	11011101	
Summe:	01011101	$= 128 - 35 = 93$

Aufgabe 2 (Aufwand und Komplexität [15 Punkte] + 3 Bonuspunkte)

a) Berechnen Sie mit Hilfe des Mastertheorems eine asymptotische obere Schranke für eine Funktion T mit folgenden Eigenschaften (2 P)

$$T(1) = 1$$
$$T(n) = 49T(n/7) + 3n^2$$

Lösung

$T(n) = 49T(n/7) + 3n^2$
$\Leftrightarrow \; a = 49, \; b = 7, \; k = 2$

Offensichtlich ist $a = b^k \; (49 = 7^2)$

Damit folgt nach dem Master-Theorem:

$T(n) \in O(n^2 \log n)$.

b) Bestimmen Sie für folgende Programmfragmente wie oft *genau* die Methode `tuWas()` aufgerufen wird. K und M sind dabei beliebige natürliche Zahlen (je 2 P):

1. ```
   for (int i=0;i<K;i++)
     for (int j=i,j>=0;j--)
       tuWas();
   ```

   Lösung

   $K * (K + 1)/2$

2. ```
   for (int i=0;i<K;i++)
     for (int j=0,j<K;j++)
       tuWas();
   ```

   Lösung

   K^2

3. ```
   for (int i=1;i<K;i++)
     for (int j=0,j<M;j++)
       tuWas();
   ```

   Lösung

   $(K - 1) * M$

c) Für natürliche Zahlen a, m bezeichnet $a \bmod m$ den Rest von a bei Division durch m. [Erinnerung: `%` steht für mod.]
Die Methode `modexp` liefert bei positiven Argumentwerten a, e und m den Wert $a^e \bmod m$ (also den Rest von a^e bei Division durch m).
   ```

```
public class modexp {
  // Voraussetzung: a,e,m > 0
  public static int modexp(int a, int e, int m) {
    int r = 1;
    while ( e > 0) {
      if ( (e % 2) == 1)
      r = ( r * a) % m;
      e = e/2;
      a = a * a % m;
    }
    return r;
  }
}
```

1. Berechnen Sie $10^{13} \bmod 7$. Sie können obigen Algorithmus verwenden. (4 P)

   Lösung

   (3P Berechnungsweg, 1P Ergebnis)

   1. Möglichkeit:
   $10^{13} = 10 \cdot (10^6)^2 = 3 \cdot (3^6)^2 \bmod 7$
   $= 3 \cdot ((3^3)^2)^2$
   $= 3 \cdot ((3 \cdot 3^2)^2)^2 \qquad (3^2 = 9 \bmod 7 = 2)$
   $= 3 \cdot ((3 \cdot 2)^2)^2$
   $= 3 \cdot (36)^2 \qquad (36 \bmod 7 = 1)$
   $= 3 \cdot 1^2$
   $= 3 \bmod 7$

   2. Möglichkeit: gemäß angegebenem Algorithmus

   - $r = 1, a = 10,\ e = 13\ > 0$ ungerade
     $\Rightarrow r = 10 \bmod 7\ = 3,\ e = 6,\ a = 100 \bmod 7\ = 2$
   - $r = 3, a = 2,\ e = 6\ > 0$ gerade
     $\Rightarrow e = 3,\ a = 4 \bmod 7\ = 4$
   - $r = 3, a = 4,\ e = 3\ > 0$ ungerade
     $\Rightarrow r = 4 \cdot 3 =\ 12\ =\ 5 \bmod 7,\ e = 1,\ a = 4 \cdot 4 = 16\ =\ 2 \bmod 7$
   - $r = 5, a = 2,\ e = 1\ > 0$ ungerade
     $\Rightarrow r = 2 \cdot 5 =\ 10\ =\ 3 \bmod 7,\ e = 0,\ a = 4\ \Rightarrow$ return 3

2. Wie viele Iterationen erfolgen höchstens, falls $e = 1000000$? Wie viele mindestens? Begründen Sie die Antwort. (3 P)

   Lösung

   e wird in jeder Iteration halbiert (ganzzahlig).
   Wie oft muss 1000000 halbiert werden, bis der Wert 0 erreicht wird ist?

   Abschätzung: $1000000 = 1000000\ < 10241024 = 2^{20}$

$\Rightarrow$ spätestens nach 20 Iterationen ist der Wert 0 erreicht.
(19 mal genügt nicht, denn $2^{19} = 512.1024 \ < 1000000$)

3. *Bonus **(3 Bonuspunkte)**: Beweisen Sie mit einer Schleifeninvariante, dass der Algorithmus korrekt ist. Sie können benutzen, dass $(x \bmod m) * (y \bmod m) = (x * y \bmod m)$ ist.

Lösung

Korrektheit: Sei $z = a^e \bmod m$

($e > 0$ und $z = r \cdot a^e \bmod m$) ist die Schleifeninvariante.

Sie gilt offenbar in der ersten Iteration.

Sei ($e > 0$ und $z = r \cdot a^e \bmod m$ vor einer Iteration

1. Fall:
e ist gerade. Dann bleibt der Wert von r erhalten, e wird halbiert, a wird quadriert.
Wegen $r \cdot a^e = r \cdot (a^2)^{\frac{e}{2}} \bmod m$ gilt die Gleichung auch nach der Iteration.

2. Fall:
e ist ungerade.
Wegen $r \cdot a^e = r \cdot a \cdot (a^2)^{\frac{e-1}{2}} \bmod m$ gilt die Gleichung auch nach der Iteration.

Aufgabe 3 (Objektorientierung & Programmierung [10 Punkte])

a) Betrachten Sie folgende Java-Klassen: (2 P)

```
class A {
  public static void foo (int x, long y) {
    System.out.println(x+y);
  }
  public static void foo (long x, int y) {
    System.out.println(x*y);
  }
}

class B {
  private byte x = 3;
  public void foo (int x, int y) {
    A.foo(this.x+y, x);
  }
  public static void main (String[] args) {
    B objB = new B();
    objB.foo(1,2);
  }
}
```

Geben Sie im obigen Beispiel alle Situationen an, in denen Methoden überladen oder überschrieben werden und Attribute überdeckt werden.

Lösung

Die Methode `foo` in der Klasse A ist überladen.
Es werden im Beispiel keine Methoden überschrieben.
In der Methode `foo` in der Klasse B überdeckt der Parameter `x` vom Typ `int` das Instanzattribut `x` der Klasse B vom Typ `byte`.

b) Erläutern Sie folgende Begriffe und geben Sie je ein Beispiel: (4 P)

- Implizite Typumwandlung
- Explizite Typumwandlung

Lösung Bei einer impliziten Typumwandlung wird ein Ausdruck von Typ `A` an einer Stelle verwendet, wo eigentlich ein Ausdruck vom Typ `B` erwartet wird (z.B. Zuweisung an Variable vom Typ `B`). Der Ausdruck wird intern in einen vom Typ `B` verwandelt. Implizite Typumwandlungen sind in der Regel nicht verlustbehaftet und erfolgen in Richtung „größerer“ Typen (z.B. von `int` nach `long`). (1 P)

```
class A {                    (1 P)
  int x;
```

```
  long y;
  // Implizite Typumwandlung int -> long
  y = x*1000;
  .
  .
}
```

Bei einer expliten Typumwandlung (*cast*) wird ein Ausdruck von Typ `A` explizit in einen Ausdruck eines „kleineren“ Typs `B` umgewandelt. Dieses ist in der Regel verlustbehaftet und muss explizit durch einen in Klammern vorangestellten Typ angezeigt werden. (1 P)

```
class A {                        (1 P)
  int x;
  long y;
  // Explizite Typumwandlung long -> int
  x = (int) y;
  .
  .
}
```

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt. (4 P)

Für jede falsche Antwort wird 1 Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt.		
Es gibt Java-Programme, die nicht terminieren.		
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		

Lösung

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt		X
Es gibt Java-Programme, die nicht terminieren.	X	
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		X
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		X

Aufgabe 4 (Java [20 Punkte])

a) Geben Sie eine einfache Klasse `Person` an, mit der Personendaten abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Folgende Operationen sollen möglich sein: (10 P)

- Erzeugen einer Instanz unter Angabe des Namens,
- Erzeugen einer Instanz unter Angabe von Name und Gewicht,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

Lösung

```
public class Person {            // Punktezusammensetzung:
  String name;                                    // 0.5 P
  long gewicht;      // oder int oder sonstwas    // 0.5 P
  static int anzahl;                     // attr: 0.5 P,
                                        // static: 0.5 P
  static long gesamtgewicht;            // attr: 0.5 P,
                                        // static: 0.5 P
  public Person(String t){                           // 1 P
    name = t;
    anzahl++;
  }

  public Person(String t, long p) {                  // 1 P
    name = t;
    anzahl++;
    gewicht = p;
    gesamtgewicht = gesamtgewicht + p;
  }

  public void setGewicht(long p) {                  // 1 P
    gesamtgewicht = gesamtgewicht - gewicht + p;
    gewicht = p;
  }

  public long getGewicht() {                        // 1 P
    return gewicht;
  }

  public static int wieviele() {          // Meth: 1 P,
    return anzahl;                       // static 0.5 P
  }

  public static long gesamtgewicht() {       // Meth: 1 P,
    return gesamtgewicht;                 // static 0.5 P
  }
}
```

b) Für natürlicher Zahlen n, k ($n \geq k \geq 0$) gibt der Binomialkoeffizient $\binom{n}{k}$ an, wieviele Möglichkeiten es gibt, aus n Elementen k auszuwählen. Hieraus ergibt sich zwangsläufig, dass in den beiden Fällen $n = k$ und $k = 0$ das Ergebnis jeweils 1 ist. Für die anderen Fälle kann bei der Berechnung die folgende Eigenschaft des Binomialkoeffizient ausgenutzt werden:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Geben Sie eine rekursive Java-Methode `binom` an, die für zwei nicht-negative Integer-Argumente n und k, mit $n \geq k \geq 0$, den Binomialkoeffizienten berechnet. (2 P)
Hinweis: In der Funktion soll nicht getestet werden, ob die Voraussetzung $n \geq k \geq 0$ erfüllt ist.

Lösung

```
static int binom (int n, int k)
{
    if (n == k || k == 0) {
        return 1;
    } else {
        return binom (n - 1, k) + binom (n - 1, k - 1);
    }
}
```

c) In folgendem Java-Codefragment sind einige Fehler enthalten. Die Zeilen des Aufretens sind durch Kommentare markiert. Lokalisieren Sie die Fehler näher, indem Sie die genauen Fehlerstellen im Programmtext markieren. Geben Sie für jeden Fehler eine kurze Fehlerbeschreibung an, die eine mögliche Meldung des Java-Systems widerspiegelt. Machen Sie insbesondere eine Aussage dazu, ob der Fehler zur Übersetzungszeit oder zur Laufzeit auftritt. Schlagen Sie vor, wie der Fehler möglichst einfach im Sinne der beabsichtigten Wirkungsweise des Programmes korrigiert werden kann. (8 P)

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.

    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)        // Fehlerzeile 1
      if (p.dat = 0)                       // Fehlerzeile 2
```

```
      cnt++;                              // Fehlerzeile 3(a)
    else
      p = p.next;
  return cnt;                             // Fehlerzeile 3(b)
}

private class Element {
  int dat;
  Element next;
}

public static void main(String[] args) {
  List lst  = new List;                   // Fehlerzeile 4
      :
  // Aufbau von lst. Dieser Code ist fehlerfrei.
      :
  lst.zeroCnt(3);
}
}
```

Lösung

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.

    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)       // Fehlerzeile 1
      if (p.dat = 0)                      // Fehlerzeile 2
        cnt++;                            // Fehlerzeile 3(a)
      else
        p = p.next;
    return cnt;                           // Fehlerzeile 3(b)
  }

  private class Element {
    int dat;
    Element next;
  }

  public static void main(String[] args) {
    List lst  = new List;                 // Fehlerzeile 4
        :
    // Aufbau von lst. Dieser Code ist fehlerfrei.
```

```
        :
    lst.zeroCnt(3);
  }
}
```

– Fehlerzeile 1: **(2 P)**
Fehlerart: *Laufzeitfehler. Zugriff auf* `null`*-Referenz.*

Korrekturvorschlag: *Vertauschen der Bedingungsreihenfolge (weil* `&&` *abhängig von Reihenfolge). Also:* `(p != null && p.dat != x)`

– Fehlerzeile 2: **(2 P)**
Fehlerart: *Compilierfehler, genauer Typfehler. Die Zuweisung liefert ein Integerresultat, wo ein Boolescher Wert erwartet wird.*

Korrekturvorschlag: *Zuweisungsoperator = durch Vergleichsoperator == ersetzen.*

– Fehlerzeile 3(a) und 3(b): **(2 P)**
Fehlerart: *Compilierfehler, genauer statischer Initialisierungsfehler. Die lokale Methodenvariable* `cnt` *ist (möglicherweise) uninitialisiert.*

Korrekturvorschlag: `cnt`*-Deklaration um Initialisierung ergänzen, also:* `int cnt = 0;`

– Fehlerzeile 4: **(2 P)**
Fehlerart: *Compilierfehler, genauer Syntaxfehler. Bei Konstruktoraufruf wird (leere) Parameterliste erwartet.*

Korrekturvorschlag: *Fehlerzeile 4:* `List lst = new List();`

Aufgabe 5 (Sortierverfahren [10 Punkte])

a) Beschreiben Sie stichpunktartig die Merkmale eines *Teile und Herrsche*-Algorithmus (Divide&Conquer). (3 P)

Lösung

Voraussetzung Problem lässt sich in unabhängige Teilprobleme zerlegen, die separat gelöst werden können und deren Lösung zu Lösung des Ausgangsproblems kombiniert werden kann.

Teile das Problem in Teilprobleme auf.

Beherrsche große Teilprobleme durch rekursives Aufteilen und kleine Teilprobleme durch direktes Lösen.

Kombiniere Lösungen der Teilprobleme zu Lösung des Ausgangsproblems.

(Wohlwollend korrigieren. Wer sich allein auf Mergesort bezieht (und dies korrekt beschreibt) bekommt auch die volle Punktzahl.)

b) Betrachten Sie folgende Implementierung der Methode `mergeSort` aus dem Vorlesungsskript:

```
public void mergeSort() {
  mergeSort(0,feld.length-1);
}
private void mergeSort(int von, int bis) {
  if (von == bis) return;
  int mitte = (von + bis) / 2; // ganzz. Division
  mergeSort(von, mitte);
  mergeSort(mitte+1, bis);
  vereinige(von,mitte,bis);
}
```

Implementieren Sie dazu die Methode `vereinige`. (7 P)

Lösung

```
private void vereinige(int von, int mitte, int bis) {
  int[] tmp = new int[bis-von+1]; // Hilfsfeld
  int i = von,j = mitte+1, k;

  // fuelle tmp sukzessive auf, benutze dazu den folgenden ..
  for (k=0; i<=mitte && j <= bis;k++) {   //.. Wechselschalter:
    tmp[k]=(feld[i] <= feld[j] ? feld[i++] : feld[j++]);
  }   // i oder j ist am Ende angekommen ..
  while ( i <= mitte) // .. => Rest noch kopieren:
    tmp[k++] = feld[i++];
    // falls j < bis, macht nichts: sind alle groesser
```

```
    // tmp[0..k-1] zurueckkopieren:
  for (int t = 0; t<k; t++)
    feld[von + t] = tmp[t];
}
```

Aufgabe 6 (Algorithmen und Datenstrukturen [15 Punkte])

a) Im folgenden Programmfragment wird mit Hilfe von `push` und `pop` ein anfangs leerer Stack gesteuert. `push` operiert wie üblich, aber `pop` soll etwas anders wirken als in der Vorlesung: Der Aufruf entfernt das zuletzt hinzugefügte Element und liefert es auch als Rückgabewert[1]. Notieren Sie in einer Tabelle, welchen Inhalt der Stack nach den einzelnen Anweisungen hat. Geben Sie auch an, welche Ausgabe das Programmstück erzeugt. (5 P)

```
push(12);
push(6);
push(pop()+pop());
push(8);
push(8);
push(64);
push(pop()/pop());      // auf Auswertungsreihenfolge achten!
push(1/(pop()/pop()));  // ..
push(3);
push(pop()*pop());      // ..
push(pop()+pop());      // ..
System.out.println(pop());
```

Lösung

					64					
				8	8	8		3		
	6		8	8	8	8	1	1	3	
12	12	18	18	18	18	18	18	18	18	21

b) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt[2]. (4 P)

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.		
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.		
Der Aufwand für Merge-Sort ist immer $O(N)$		
Ein Stack arbeitet nach dem FIFO-Prinzip		

Lösung

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.	X	
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.	X	
Der Aufwand für Merge-Sort ist immer $O(N)$		X
Ein Stack arbeitet nach dem FIFO-Prinzip		X

[1] In der Vorlesung war für Letzteres der Aufruf `top()` zuständig.

[2] **Für jede falsche Antwort wird 1 Punkt abgezogen!** Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

c) Ein Algorithmus heißt

- *deterministisch*, falls er zu jeder Eingabe und jedem Zustand der Berechnung einen eindeutig bestimmten Folgezustand definiert,
- *terminierend*, falls er bei jeder Eingabe nach endlich vielen Schritten einen Endzustand erreicht (d.h. abbricht) und
- *determiniert*, falls jede Berechnung des Algorithmus bei gleichen Eingaben das gleiche Ergebnis (d.h. die gleiche Ausgabe) liefert[3].

Geben Sie für die beiden folgenden Algorithmen an, welche der drei genannten Eigenschaften sie haben und welche nicht und begründen Sie die Antwort stichpunktartig. (6 P)

Algorithmus 1:

1. Wähle eine beliebige Zahl zwischen -10 und 20.
2. Multipliziere diese Zahl mit sich selbst.
3. Gib das Ergebnis aus, falls es größer als 10 ist. Andernfalls gehe zu 2.

Algorithmus 2:

```
public float rechne (float a) {
  float p;
  if (a>10) {
    p = Math.random(); // erzeugt Zufallszahl zwischen 0 und 1
    if (a>20) {
      a = a-p;
    }
    a = a+p;
  }
  return a;
}
```

Lösung

Algorithmus 1: (3 P)
nicht-deterministisch (ich rate 3 du rätst 4), nicht-terminierend (Endlosschleife z.B. bei 0), nicht-determiniert (wenn er hält, dann zwar immer mit der gleichen Ausgabe — aber er hält eben nicht immer)

Algorithmus 2: (3 P)
nicht-deterministisch (da konkrete Berechnung vom Zufall abhängig), terminierend (da keine Schleife), nicht-determiniert (z.B. $a = 11$ und $a = 21$ liefern unterschiedliche Ergebnisse (sofern $p \neq 0$)).

[3] man sollte vielleicht besser sagen: das Ergebnis des Algorithmus ist determiniert

Achtung: Auf die Begründung kommt es an! Bei Algorithmus 2 wäre z.B. auch die Antwort „deterministisch“ korrekt, falls zusätzlich bemerkt wurde, dass Math.random() bei gleichem Startwert des Pseudozufallsgenerators gleiche Ergebnisse liefert.

Wiederholungsklausur Informatik I
Sommersemester 2007
Prof. Dr. Carsten Damm
19. April 2007, 12:00 - 13:30 Uhr
Bearbeitungszeit: 90 Minuten

Vorname: Tiger

Nachname: Scott

Matrikelnummer: 0815

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

	Punkte	erreicht
Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung)	20	
Aufgabe 2 (Aufwand und Komplexität + 3 Bonuspunkte)	15	
Aufgabe 3 (Objektorientierung & Programmierung)	10	
Aufgabe 4 (Java)	20	
Aufgabe 5 (Sortierverfahren)	10	
Aufgabe 6 (Algorithmen und Datenstrukturen)	15	
Summe	90	

Note:

Name: Tiger Scott MatNr.: 0815

Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung [20 Punkte])

a) Geben Sie einen regulären Ausdruck über dem Alphabet $\{a, f, g, s, v, z\}$ an, der das übliche Verhalten von Passagieren in einem Flughafen beschreibt.

Die Aktionen sind: Gepäckwagen nehmen: g, Gepäckwagen zurückgeben: z, Flughafen verlassen: v, Gepäckwagen schieben: s, Gepäckstück am Schalter abgeben: a, mit Flugzeug abreisen: f.

Beispiel $gsssaaaszf$: Passagier schiebt Gepäckwagen mehrfach, gibt drei Gepäckstücke am Schalter ab, gibt den Gepäckwagen ab und fliegt ab.

Vorgaben: Eingang, Ausgang und Schalter sind voneinander entfernt. Wenn Gepäckstücke am Schalter nicht abgegeben werden (Reisepaß abgelaufen), wird der Gepäckwagen zurückgegeben und der Flughafen verlassen. (6 P)

b) Entwerfen Sie einen endlichen Automaten der die folgende Sprache L über dem Alphabet $\{1, 2, ., E\}$ akzeptiert (Zustandsdiagramm genügt).

$$L = (\{1,2\}\{1,2\}^*.\{1,2\}^*)(E\{1,2\}\{1,2\}^* \cup \{\varepsilon\})$$

Erläuterung: L besteht aus den Gleitkommaliteralen, die mit einer Ziffer beginnen, einen Dezimalpunkt enthalten und außer diesem und einem eventuellen Exponentenzeichen nur die Ziffern 1 und 2 (z.B. 12.12 oder 12.E2, aber nicht 123.123 oder .11E12 oder 12.E-1): (6 P)

c) Gegeben sind die Zahlen 128 und 35. Ihre Summe ist 163.

1. Stellen Sie beide als 8-Bit Binärzahlen dar. (3 P)
2. Addieren Sie beide Zahlen im Binärformat und verifizieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (2 P)
3. Stellen Sie -35 als Zweierkomplement dar und berechnen Sie $128 - 35$. (3 P)

Name: Tiger Scott MatNr.: 0815

Aufgabe 2 (Aufwand und Komplexität [15 Punkte] + 3 Bonuspunkte)

a) Berechnen Sie mit Hilfe des Mastertheorems eine asymptotische obere Schranke für eine Funktion T mit folgenden Eigenschaften (2 P)

$$T(1) = 1$$
$$T(n) = 49T(n/7) + 3n^2$$

b) Bestimmen Sie für folgende Programmfragmente wie oft *genau* die Methode `tuWas()` aufgerufen wird. K und M sind dabei beliebige natürliche Zahlen (je 2 P):

1. ```
   for (int i=0;i<K;i++)
     for (int j=i,j>=0;j--)
       tuWas();
   ```
2. ```
   for (int i=0;i<K;i++)
     for (int j=0,j<K;j++)
       tuWas();
   ```
3. ```
   for (int i=1;i<K;i++)
     for (int j=0,j<M;j++)
       tuWas();
   ```

c) Für natürliche Zahlen a, m bezeichnet $a \bmod m$ den Rest von a bei Division durch m. [Erinnerung: `%` steht für mod .]
Die Methode `modexp` liefert bei positiven Argumentwerten a, e und m den Wert $a^e \bmod m$ (also den Rest von a^e bei Division durch m).

```
public class modexp {
  // Voraussetzung: a,e,m > 0
  public static int modexp(int a, int e, int m) {
    int r = 1;
    while ( e > 0) {
      if ( (e % 2) == 1)
      r = ( r * a) % m;
      e = e/2;
      a = a * a % m;
    }
    return r;
  }
}
```

1. Berechnen Sie $10^{13} \bmod 7$. Sie können obigen Algorithmus verwenden. (4 P)
2. Wie viele Iterationen erfolgen höchstens, falls $e = 1000000$? Wie viele mindestens? Begründen Sie die Antwort. (3 P)
3. *Bonus **(3 Bonuspunkte)**: Beweisen Sie mit einer Schleifeninvariante, dass der Algorithmus korrekt ist. Sie können benutzen, dass $(x \bmod m) * (y \bmod m) = (x * y \bmod m)$ ist.
```

Name: Tiger Scott MatNr.: 0815

Aufgabe 3 (Objektorientierung & Programmierung [10 Punkte])

a) Betrachten Sie folgende Java-Klassen: (2 P)

```
class A {
  public static void foo (int x, long y) {
    System.out.println(x+y);
  }
  public static void foo (long x, int y) {
    System.out.println(x*y);
  }
}

class B {
  private byte x = 3;
  public void foo (int x, int y) {
    A.foo(this.x+y, x);
  }
  public static void main (String[] args) {
    B objB = new B();
    objB.foo(1,2);
  }
}
```

Geben Sie im obigen Beispiel alle Situationen an, in denen Methoden überladen oder überschrieben werden und Attribute überdeckt werden.

b) Erläutern Sie folgende Begriffe und geben Sie je ein Beispiel: (4 P)

- Implizite Typumwandlung
- Explizite Typumwandlung

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt. (4 P)

Für jede falsche Antwort wird 1 Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt.		
Es gibt Java-Programme, die nicht terminieren.		
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		

Name: Tiger Scott MatNr.: 0815

Aufgabe 4 (Java [20 Punkte])

a) Geben Sie eine einfache Klasse `Person` an, mit der Personendaten abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Folgende Operationen sollen möglich sein: (10 P)

- Erzeugen einer Instanz unter Angabe des Namens,
- Erzeugen einer Instanz unter Angabe von Name und Gewicht,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

b) Für natürlicher Zahlen n, k ($n \geq k \geq 0$) gibt der Binomialkoeffizient $\binom{n}{k}$ an, wieviele Möglichkeiten es gibt, aus n Elementen k auszuwählen. Hieraus ergibt sich zwangsläufig, dass in den beiden Fällen $n = k$ und $k = 0$ das Ergebnis jeweils 1 ist. Für die anderen Fälle kann bei der Berechnung die folgende Eigenschaft des Binomialkoeffizient ausgenutzt werden:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Geben Sie eine rekursive Java-Methode `binom` an, die für zwei nicht-negative Integer-Argumente n und k, mit $n \geq k \geq 0$, den Binomialkoeffizienten berechnet. (2 P)
Hinweis: In der Funktion soll nicht getestet werden, ob die Voraussetzung $n \geq k \geq 0$ erfüllt ist.

c) In folgendem Java-Codefragment sind einige Fehler enthalten. Die Zeilen des Aufretens sind durch Kommentare markiert. Lokalisieren Sie die Fehler näher, indem Sie die genauen Fehlerstellen im Programmtext markieren. Geben Sie für jeden Fehler eine kurze Fehlerbeschreibung an, die eine mögliche Meldung des Java-Systems widerspiegelt. Machen Sie insbesondere eine Aussage dazu, ob der Fehler zur Übersetzungszeit oder zur Laufzeit auftritt. Schlagen Sie vor, wie der Fehler möglichst einfach im Sinne der beabsichtigten Wirkungsweise des Programmes korrigiert werden kann. (8 P)

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.
```

```
    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)        // Fehlerzeile 1
      if (p.dat = 0)                       // Fehlerzeile 2
        cnt++;                             // Fehlerzeile 3(a)
      else
        p = p.next;
    return cnt;                            // Fehlerzeile 3(b)
  }

  private class Element {
    int dat;
    Element next;
  }

  public static void main(String[] args) {
    List lst  = new List;                 // Fehlerzeile 4
        :
    // Aufbau von lst. Dieser Code ist fehlerfrei.
        :
    lst.zeroCnt(3);
  }
}
```

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815

Aufgabe 5 (Sortierverfahren [10 Punkte])

a) Beschreiben Sie stichpunktartig die Merkmale eines *Teile und Herrsche*-Algorithmus (Divide&Conquer). (3 P)

b) Betrachten Sie folgende Implementierung der Methode `mergeSort` aus dem Vorlesungsskript:

```
public void mergeSort() {
  mergeSort(0,feld.length-1);
}
private void mergeSort(int von, int bis) {
  if (von == bis) return;
  int mitte = (von + bis) / 2; // ganzz. Division
  mergeSort(von, mitte);
  mergeSort(mitte+1, bis);
  vereinige(von,mitte,bis);
}
```

Implementieren Sie dazu die Methode `vereinige`. (7 P)

Name: Tiger Scott MatNr.: 0815

Aufgabe 6 (Algorithmen und Datenstrukturen [15 Punkte])

a) Im folgenden Programmfragment wird mit Hilfe von `push` und `pop` ein anfangs leerer Stack gesteuert. `push` operiert wie üblich, aber `pop` soll etwas anders wirken als in der Vorlesung: Der Aufruf entfernt das zuletzt hinzugefügte Element und liefert es auch als Rückgabewert[1]. Notieren Sie in einer Tabelle, welchen Inhalt der Stack nach den einzelnen Anweisungen hat.

Geben Sie auch an, welche Ausgabe das Programmstück erzeugt. (5 P)

```
push(12);
push(6);
push(pop()+pop());
push(8);
push(8);
push(64);
push(pop()/pop());      // auf Auswertungsreihenfolge achten!
push(1/(pop()/pop()));  // ..
push(3);
push(pop()*pop());      // ..
push(pop()+pop());      // ..
System.out.println(pop());
```

b) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt[2]. (4 P)

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.		
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.		
Der Aufwand für Merge-Sort ist immer $O(N)$		
Ein Stack arbeitet nach dem FIFO-Prinzip		

c) Ein Algorithmus heißt

- *deterministisch*, falls er zu jeder Eingabe und jedem Zustand der Berechnung einen eindeutig bestimmten Folgezustand definiert,
- *terminierend*, falls er bei jeder Eingabe nach endlich vielen Schritten einen Endzustand erreicht (d.h. abbricht) und
- *determiniert*, falls jede Berechnung des Algorithmus bei gleichen Eingaben das gleiche Ergebnis (d.h. die gleiche Ausgabe) liefert[3].

[1] In der Vorlesung war für Letzteres der Aufruf `top()` zuständig.

[2] **Für jede falsche Antwort wird 1 Punkt abgezogen!** Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

[3] man sollte vielleicht besser sagen: das Ergebnis des Algorithmus ist determiniert

Geben Sie für die beiden folgenden Algorithmen an, welche der drei genannten Eigenschaften sie haben und welche nicht und begründen Sie die Antwort stichpunktartig. (6 P)

Algorithmus 1:

1. Wähle eine beliebige Zahl zwischen -10 und 20.
2. Multipliziere diese Zahl mit sich selbst.
3. Gib das Ergebnis aus, falls es größer als 10 ist. Andernfalls gehe zu 2.

Algorithmus 2:

```
public float rechne (float a) {
  float p;
  if (a>10) {
    p = Math.random(); // erzeugt Zufallszahl zwischen 0 und 1
    if (a>20) {
      a = a-p;
    }
    a = a+p;
  }
  return a;
}
```

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815